

Data Collection, Cleaning, and Preprocessing for Logistics Analysis

DataCo Smart Supply Chain Dataset

Executive Summary

This report presents a practical preprocessing pipeline applied to the actual DataCo Smart Supply Chain CSV. Unlike a purely theoretical description, the report records measured data-quality results from the raw file, documents every major cleaning decision, provides reproducible Python code, and creates derived logistics features and scaled variables. The goal is to transform raw supply-chain records into an analysis-ready dataset while preserving genuine business observations.

Metric	Measured result
Raw records	180,519
Raw columns	53
Raw missing cells	336,209
Exact duplicate rows	0
Final records	180,519
Final columns	56
Final missing cells	0
Final exact duplicates	0

1. Problem Statement

Logistics organizations generate large volumes of order, customer, product, transportation, and delivery data. Before such data can support reliable decisions, it must be checked for completeness, consistency, validity, and suitable numerical representation. The problem addressed in this task is to design and demonstrate a reproducible preprocessing pipeline for supply-chain data, identify data-quality issues, apply justified treatments, and validate the final dataset.

2. Objectives

- Simulate the collection of integrated logistics data from operational systems.
- Profile the raw dataset using Python and pandas.
- Measure missing values, duplicate records, data types, and suspicious values.
- Apply justified missing-value and data-cleaning strategies.
- Detect outliers with the Interquartile Range (IQR) method.
- Create logistics features such as shipping delay and delay category.
- Normalize and standardize selected numerical variables.
- Protect unnecessary personal/sensitive fields from the analysis dataset.
- Validate the final dataset and compare quality before and after preprocessing.
- Prepare a reproducible Python script and cleaned CSV for further analysis.

3. Dataset Selection and Justification

The DataCo Smart Supply Chain Dataset was selected because it directly represents supply-chain and logistics processes. It contains order, customer, product, market, shipping, delivery, sales, and operational variables. It is therefore suitable for the assignment requirements involving missing data, outliers, normalization, preprocessing, and logistics interpretation.

Dataset characteristic	Observed result
Rows	180,519
Columns	53
Date range	2015-01-01 to 2018-01-31
Shipping modes	Standard Class, First Class, Second Class, Same Day
Markets	Pacific Asia, USCA, Africa, Europe, LATAM
Customer segments	Consumer, Home Office, Corporate

Source reference: DataCo Smart Supply Chain Dataset, Mendeley Data, Version 5. The supplied ZIP also contains a variable-description file; the main analysis file used here is DataCoSupplyChainDataset.csv.

4. Data Collection Simulation

For the simulation, the CSV is treated as an integrated extract from a logistics organization. Each operational source contributes a different part of the supply-chain record.

Simulated source	Example fields	Business purpose
Order management	Order Id, order date, order status	Order tracking and transaction analysis
Transportation	Shipping Mode, shipping date, Days for shipping (real)	Transport and shipping performance
Delivery	Delivery Status, Late_delivery_risk	Service-level and delay analysis
Product catalog	Product Name, Product Price, Category Name	Product and category analysis
Customer system	Customer Id, Customer Segment, location	Customer segmentation and service analysis
Sales/finance	Sales, Benefit per order, Order Profit Per Order	Revenue and profitability analysis

5. Raw Data Profiling – Evidence from the Actual CSV

The first profiling step records the exact structure of the supplied CSV. This is important because preprocessing decisions should be based on observed data rather than assumptions.

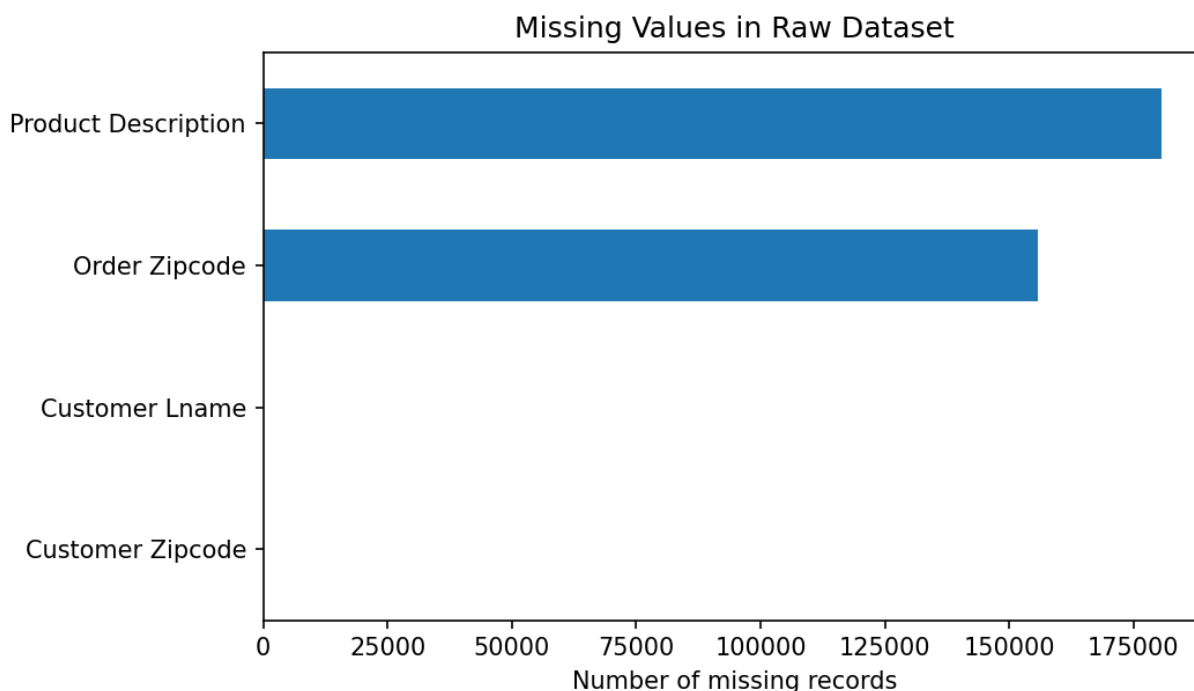
```
import pandas as pd

df = pd.read_csv("DataCoSupplyChainDataset.csv", encoding="latin1")
print(df.shape)
print(df.info())
print(df.isna().sum().sort_values(ascending=False))
print("Exact duplicates:", df.duplicated().sum())
```

Column	Missing records	Missing %
Product Description	180,519	100.00%
Order Zipcode	155,679	86.24%
Customer Lname	8	0.00%

Customer Zipcode	3	0.00%
------------------	---	-------

The most important finding is that Product Description is completely missing, while Order Zipcode is missing for 155,679 of 180,519 records (86.24%). These columns are not appropriate for direct imputation because there is too little usable information.



6. Data Cleaning Decisions and Justification

Issue / field	Observed evidence	Treatment	Reason
Product Description	180,519 missing (100%)	Drop column	No information is available to recover.
Order Zipcode	155,679 missing (86.24%)	Drop column	Very high missingness makes direct imputation unreliable.
Customer Lname	8 missing	Fill with Unknown, then exclude from analysis output	Small missing count; not needed for analytics.
Customer Zipcode	3 missing	Represent missing values as Unknown	Avoid inventing a geographic location using a numerical median.
Exact duplicate rows	0	No deletion required	No exact duplicates were found.
Dates	Stored as object/text	Convert to datetime	Enables reliable date calculations.
Categorical text	Potential spaces/case differences	Strip whitespace	Improves consistent grouping/filtering.
PII/unneeded fields	Email, password, street, names	Exclude from analysis dataset	Reduces unnecessary personal-data exposure and improves analytical focus.

Product Status	Single observed value: 0	Exclude from analysis dataset	Zero variance provides no analytical discrimination.
----------------	--------------------------	-------------------------------	--

7. Missing-Value Handling

Missing-value treatment is variable-specific. A missing product description cannot be meaningfully reconstructed because every row is missing it. In contrast, the three missing customer ZIP codes are retained as “Unknown” rather than replacing them with a fabricated geographic value. This preserves the distinction between a known and an unknown location.

```
missing_before = df.isna().sum()
print(missing_before[missing_before > 0])

df["Customer Zipcode"] = df["Customer Zipcode"].astype("string").fillna("Unknown")
```

8. Duplicate Analysis

The actual file contains 0 exact duplicate rows. Therefore, no exact duplicate records were removed. An important data-model observation is that Order Id is not unique because one order may contain multiple order items. Order Item Id, however, is unique in the supplied data. This prevents an incorrect assumption that repeated Order Id values are duplicates.

Identifier	Unique values	Duplicate IDs/rows	Interpretation
Order Id	65,752	114,767	One order can contain multiple items
Order Item Id	180,519	0	Unique order-item identifier

9. Data-Type Correction

The two operational date fields were initially read as object/text values. They were converted to datetime. Selected numeric fields were also explicitly converted where required. This allows accurate date differences, aggregation, comparisons, and validation.

```
for c in ["order date (DateOrders)", "shipping date (DateOrders)"]:
    df[c] = pd.to_datetime(df[c], errors="coerce")

df["Order Item Quantity"] = pd.to_numeric(
    df["Order Item Quantity"], errors="coerce"
)
```

Validation result: both date columns parse without invalid dates in the raw dataset. The shipping date is not earlier than the order date in the supplied records.

10. Text Standardization

Text fields are stripped of leading and trailing spaces and represented consistently. This is especially important for fields used in group-by operations, such as Shipping Mode, Market, Customer Segment, Delivery Status, and Order Status.

```
for c in ["Shipping Mode", "Market", "Customer Segment", "Delivery Status"]:
    df[c] = df[c].astype("string").str.strip()
```

11. Business-Rule Validation

Business rule	Violations found	Action
Order Item Quantity <= 0	0	No automatic deletion; investigate and document
Sales <= 0	0	No automatic deletion; investigate and document
Product price <= 0	0	No automatic deletion; investigate and document
Discount rate outside 0–1	0	No automatic deletion; investigate and document
Shipping date before order date	0	No automatic deletion; investigate and document

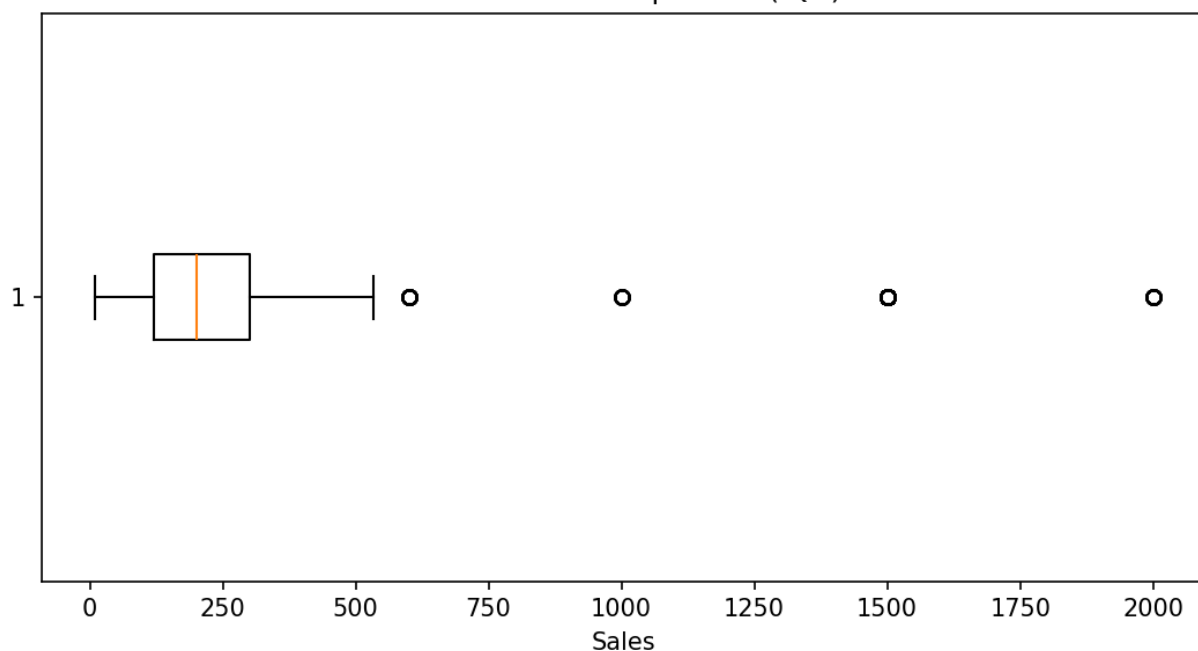
These checks show that quantity, sales, product price, discount rate, and date ordering satisfy their basic logical constraints in the supplied file. This is stronger evidence than simply stating that invalid values were checked.

12. Outlier Detection Using IQR

The IQR method was used on selected continuous variables. The purpose is to flag unusual observations for investigation, not to automatically delete legitimate logistics events.

Variable	Q1	Q3	Lower fence	Upper fence	Flagged	%
Sales	119.98	299.95	-149.98	569.91	488	0.27%
Order Item Product Price	50.00	199.99	-174.99	424.98	2,048	1.13%
Order Item Total	104.38	247.40	-110.15	461.93	1,943	1.08%
Order Profit Per Order	7.00	64.80	-79.70	151.50	18,942	10.49%
Order Item Discount	5.40	29.99	-31.48	66.87	7,537	4.18%

Sales: Outlier Inspection (IQR)



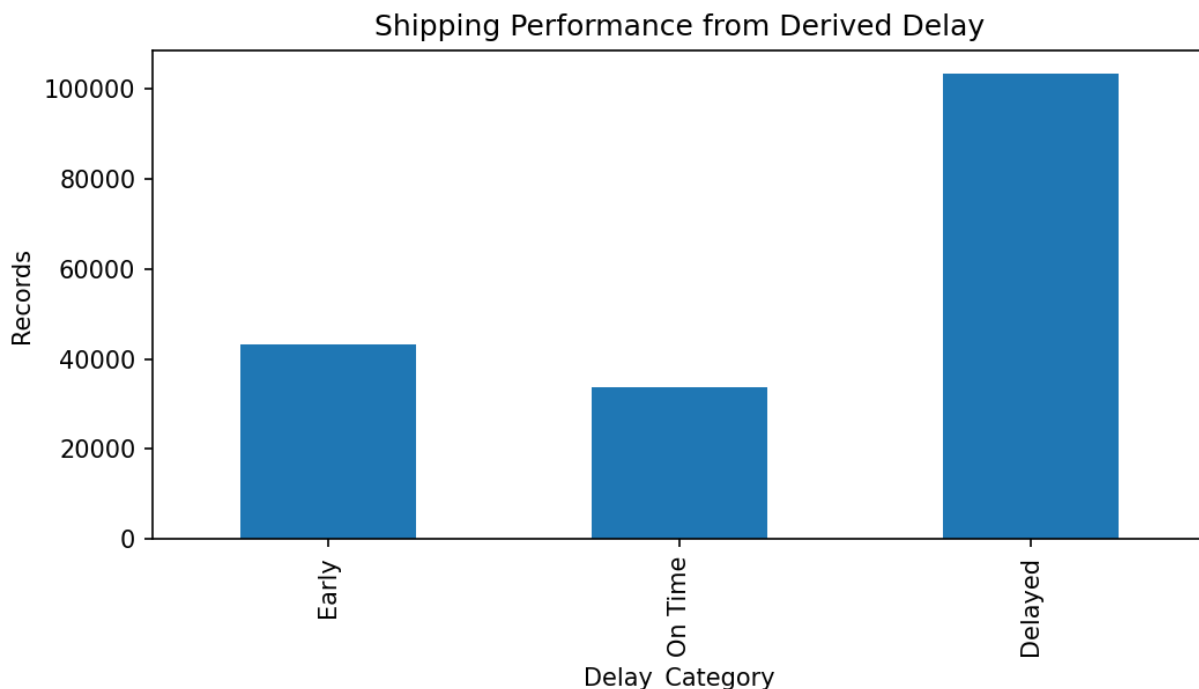
Example interpretation: Sales has 488 IQR-flagged observations (0.27%). These records are not automatically deleted because a high-value order can be genuine. Instead, the cleaned dataset stores an outlier flag so analysts can include or exclude these observations according to the business question.

13. Feature Engineering: Shipping Delay

A useful logistics feature is the difference between actual shipping duration and scheduled shipping duration. This transforms two existing variables into a direct operational performance measure.

```
df["Shipping_Delay_Days"] = (  
    df["Days for shipping (real)"]  
    - df["Days for shipment (scheduled)"]  
)  
  
df["Delay_Category"] = np.select(  
    [df["Shipping_Delay_Days"] < 0,  
     df["Shipping_Delay_Days"] == 0,  
     df["Shipping_Delay_Days"] > 0],  
    ["Early", "On Time", "Delayed"],  
    default="Unknown"  
)
```

Delay category	Records	Percentage
Early	43,366	24.02%
On Time	33,753	18.70%
Delayed	103,400	57.28%



14. Normalization and Standardization

Sales and product price were transformed using Min-Max normalization, while actual shipping days were standardized using a z-score transformation. These transformed fields are added as new variables so the original business values remain available.

```

from sklearn.preprocessing import MinMaxScaler, StandardScaler

minmax = MinMaxScaler()
df["Sales_MinMax"] = minmax.fit_transform(df[["Sales"]])
df["Product_Price_MinMax"] = minmax.fit_transform(
    df[["Order Item Product Price"]]
)

standard = StandardScaler()
df["Actual_Shipping_Days_Z"] = standard.fit_transform(
    df[["Days for shipping (real)"]]
)

```

Transformation	Variable	Purpose
Min-Max	Sales	Scale to approximately 0–1
Min-Max	Order Item Product Price	Make price comparable on a common scale
Z-score	Days for shipping (real)	Center and scale shipping duration

For a future machine-learning project, the scaler should be fitted only on the training set and then applied to validation/test data. That prevents information leakage.

15. Privacy-Aware Data Preparation

The raw dataset includes fields such as customer email, password, street, and personal names. These fields are not necessary for the Week 2 logistics analytics objective. They are excluded from the analysis-ready dataset. This is both a data-quality and responsible-data-handling decision: analysts should minimize the use of personal information when it is not required.

16. Final Validation and Before/After Results

Measure	Before	After	Interpretation
Records	180,519	180,519	No records removed solely for outliers; exact duplicates were zero.
Columns	53	56	Analysis dataset removes unusable, sensitive, and zero-variance fields and adds engineered fields.
Missing cells	336,209	0	High-missing and small missing fields are handled according to documented rules.
Exact duplicates	0	0	No exact duplicate rows were present.
Date fields	Object/text	Datetime	Supports reliable date operations.
Delay feature	Not present	Added	Supports shipping-performance analysis.
Scaled features	Not present	Added	Supports comparable numerical inputs.

```

print("Final shape:", df.shape)
print("Remaining missing cells:", df.isna().sum().sum())
print("Remaining exact duplicates:", df.duplicated().sum())
print(df.dtypes)

```

17. Exploratory Visual Evidence

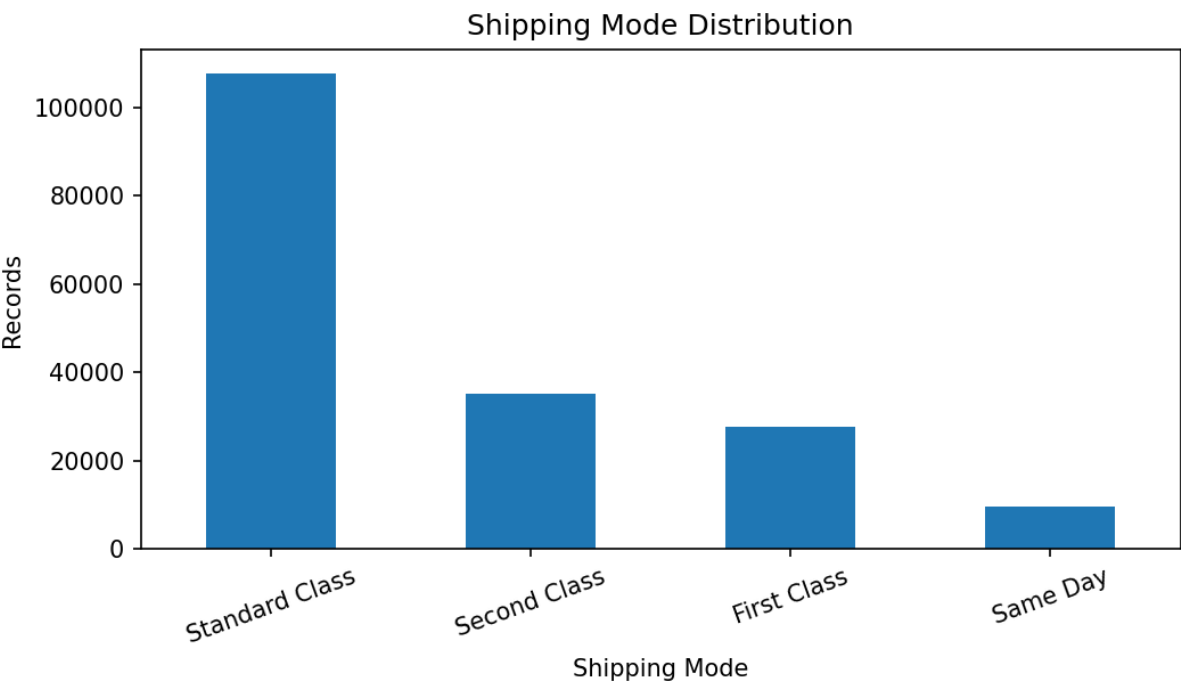
The following charts are generated from the supplied dataset after the preprocessing workflow. They provide visual evidence that the cleaned dataset can support meaningful logistics analysis.

Delivery Status



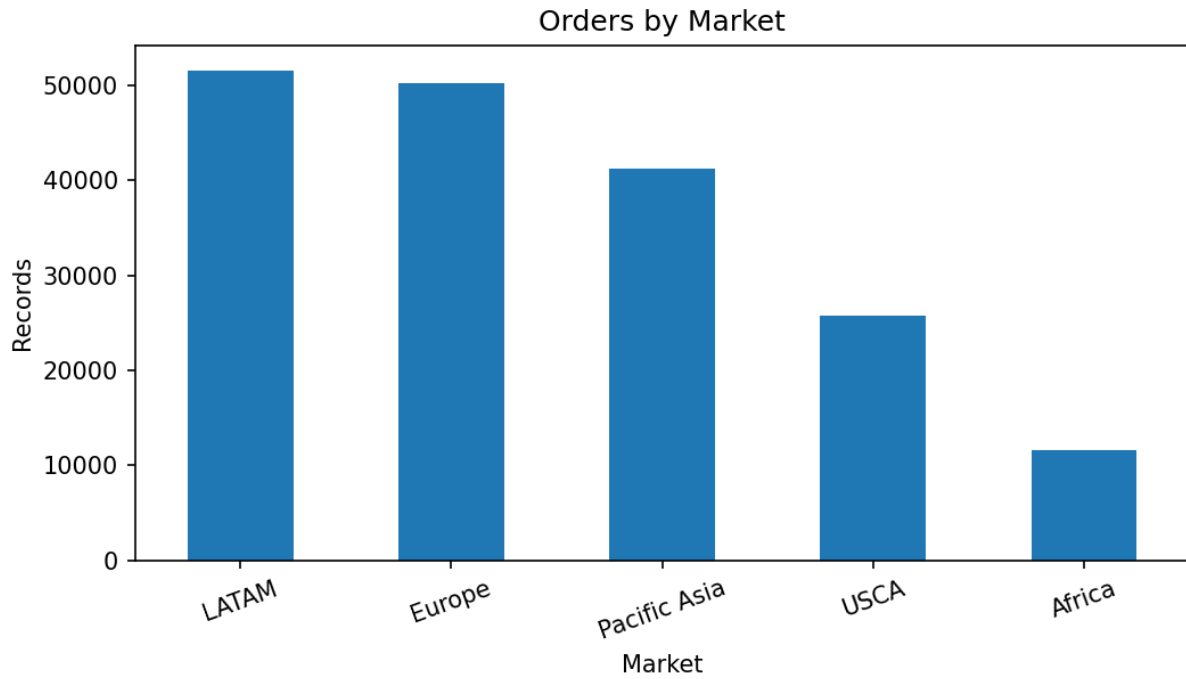
Distribution of operational delivery status.

Shipping Mode



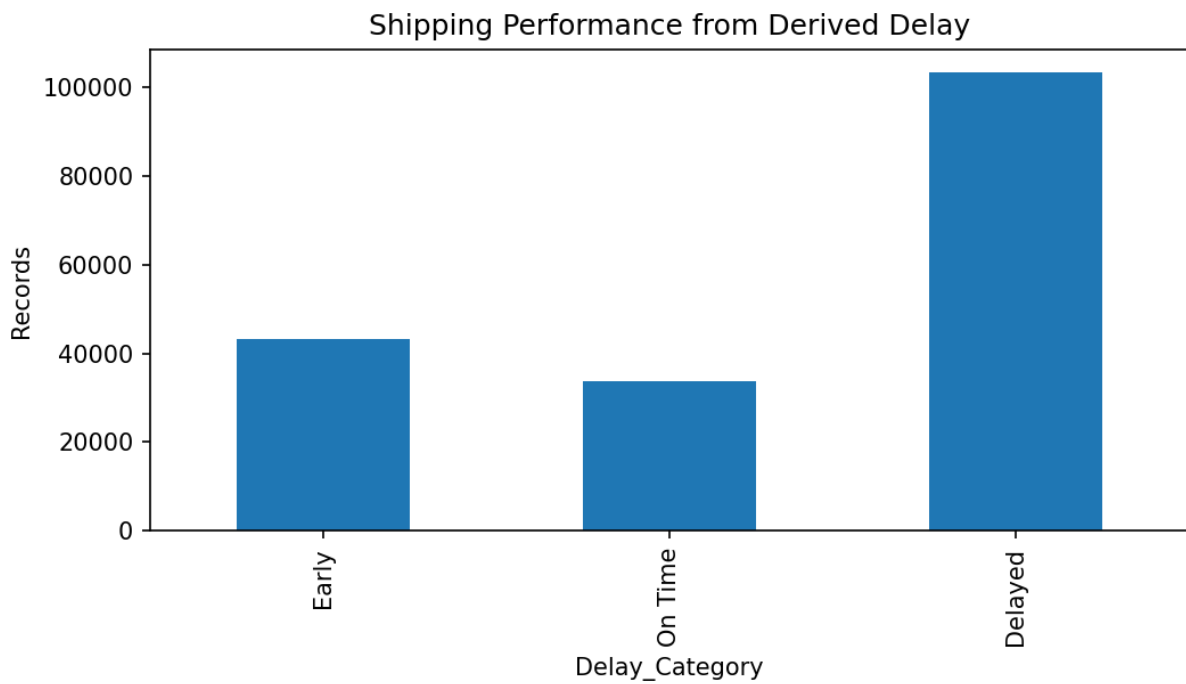
Distribution of transportation/shipping modes.

Market



Order distribution across markets.

Derived Delay Category



Operational shipping performance derived from actual versus scheduled duration.

18. Reproducible Python Preprocessing Script

The complete executable script is also supplied as a separate file named `Week_2_Data_Preprocessing.py`. The core workflow is shown below.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler

INPUT = "DataCoSupplyChainDataset.csv"
OUTPUT = "DataCoSupplyChain_Cleaned.csv"
df = pd.read_csv(INPUT, encoding="latin1")

# 1. Convert dates and numeric fields
for c in ["order date (DateOrders)", "shipping date (DateOrders)"]:
    df[c] = pd.to_datetime(df[c], errors="coerce")

# 2. Standardize text
text_cols = ["Type", "Delivery Status", "Category Name", "Customer City",
             "Customer Country", "Customer Segment", "Customer State",
             "Department Name", "Market", "Order City", "Order Country",
             "Order Region", "Order State", "Order Status", "Product Name",
             "Shipping Mode"]
for c in text_cols:
    df[c] = df[c].astype("string").str.strip()

# 3. Handle small missing fields
df["Customer Zipcode"] = df["Customer Zipcode"].astype("string").fillna("Unknown")

# 4. Remove fields unsuitable for analysis
drop_cols = ["Product Description", "Order Zipcode", "Customer Password",
             "Customer Email", "Customer Street", "Customer Fname",
             "Customer Lname", "Product Image", "Product Status"]
df = df.drop(columns=[c for c in drop_cols if c in df.columns])

# 5. Remove exact duplicate rows
df = df.drop_duplicates()

# 6. Feature engineering
df["Shipping_Delay_Days"] = (
    df["Days for shipping (real)"] - df["Days for shipment (scheduled)"]
)
df["Delay_Category"] = np.select(
    [df["Shipping_Delay_Days"] < 0,
     df["Shipping_Delay_Days"] == 0,
     df["Shipping_Delay_Days"] > 0],
    ["Early", "On Time", "Delayed"],
    default="Unknown"
)

# 7. IQR outlier flags (retain observations for business investigation)
for c in ["Sales", "Order Item Product Price", "Order Item Total",
         "Order Profit Per Order", "Order Item Discount"]:
    q1 = df[c].quantile(0.25)
    q3 = df[c].quantile(0.75)
    iqr = q3 - q1
    low = q1 - 1.5 * iqr
    high = q3 + 1.5 * iqr
    df[c + "_Outlier_Flag"] = (df[c] < low) | (df[c] > high)

# 8. Scaling
minmax = MinMaxScaler()
df["Sales_MinMax"] = minmax.fit_transform(df[["Sales"]])
df["Product_Price_MinMax"] = minmax.fit_transform(df[["Order Item Product Price"]])
```

```

standard = StandardScaler()
df["Actual_Shipping_Days_Z"] = standard.fit_transform(
    df[["Days for shipping (real)"]]
)

# 9. Validation
print("Final shape:", df.shape)
print("Remaining missing values:", df.isna().sum().sum())
print("Remaining exact duplicates:", df.duplicated().sum())

df.to_csv(OUTPUT, index=False)
print("Saved:", OUTPUT)

```

19. How the Cleaned Data Supports Logistics Decisions

Area	Possible KPI / analysis	Data fields used
Delivery performance	Delay rate, average actual shipping days	Days for shipping (real), scheduled days, Delay_Category
Transportation	Shipping-mode distribution and performance	Shipping Mode, Shipping_Delay_Days
Sales	Revenue and order-value analysis	Sales, Order Item Total, Sales per customer
Customer analysis	Segment and market distribution	Customer Segment, Market, Customer Id
Operations	Order quantity and product performance	Order Item Quantity, Product Name, Category Name
Risk	Late-delivery risk distribution	Late_delivery_risk, Delivery Status

20. Reflection

The practical analysis shows that preprocessing is not simply a sequence of commands. Each cleaning operation requires a reason. For example, dropping a 100%-missing column is justified, while filling a geographic ZIP code with the median could create a false location. Similarly, IQR flags identify unusual observations but do not prove that the records are wrong. Understanding the logistics meaning of each field is therefore essential.

Another important learning outcome is that the raw data model must be understood before deduplication. Repeated Order Id values are not necessarily duplicate transactions because a single order can contain multiple order items. Using Order Item Id as the item-level identifier prevents accidental deletion of valid records.

21. Limitations and Future Improvements

- The dataset is historical rather than a live streaming logistics feed.
- Outlier flags should be reviewed with business rules before any production deletion decision.
- Future work could add route distance, warehouse capacity, inventory levels, and carrier information.
- For predictive modeling, preprocessing should be placed inside a train/validation/test pipeline to avoid leakage.
- Additional data-quality checks could validate relationships between order totals, prices, quantities, discounts, and profit fields.

22. Alignment with Evaluation Criteria

Evaluation criterion	How this report addresses it
----------------------	------------------------------

Thoroughness of cleaning	Profiles missing values, duplicates, data types, business-rule violations, outliers, text, privacy fields, and scaling.
Clarity of issue identification	Provides measured counts, percentages, rules, tables, and explanations for each treatment.
Python code effectiveness	Includes executable pandas, NumPy, and scikit-learn examples plus a separate reproducible script.
Professional structure	Uses title page, executive summary, numbered sections, tables, charts, methodology, results, reflection, limitations, and references.
Theory + practical application	Explains why methods are used and demonstrates them on the actual DataCo CSV.
Evidence of results	Includes real row/column counts, missing-value counts, duplicate results, outlier counts, and derived logistics metrics.

23. Conclusion

The DataCo Smart Supply Chain Dataset is suitable for a comprehensive Week 2 logistics preprocessing project. The improved workflow moves beyond a generic description by applying documented cleaning decisions to the actual CSV and reporting measured results. Missing information is handled according to its meaning, exact duplicates are verified, dates are converted, text is standardized, business rules are checked, outliers are flagged rather than blindly deleted, and logistics-specific features are created. The final dataset is therefore prepared for further analytics while retaining a transparent record of the preprocessing decisions.

24. Submission Package

File	Purpose
Week_2_Logistics_High_Scoring_Report.docx	Main academic report
DataCoSupplyChainDataset.csv	Original/raw dataset
DataCoSupplyChain_Cleaned.csv	Preprocessed analysis-ready dataset
Week_2_Data_Preprocessing.py	Reproducible preprocessing code
week2_charts/	Evidence charts used in report

25. References

1. DataCo Smart Supply Chain Dataset, Mendeley Data, Version 5: <https://data.mendeley.com/datasets/8gx2fvg2k6/5>
2. pandas documentation: <https://pandas.pydata.org/docs/>
3. scikit-learn preprocessing documentation: <https://scikit-learn.org/stable/modules/preprocessing.html>
4. Python documentation: <https://docs.python.org/3/>